Static and Molecular-Dynamics-Derived Residue Graphs for Protein–Protein Binding Affinity Prediction

Student Name

The thesis is submitted to University College Dublin
in part fulfilment of the requirements for the degree of
BSc Applied and Computational Mathematics

School of Mathematics and Statistics
and
Supervisor's School (if different)
University College Dublin

Supervisor: Supervisor Name
Second Supervisor Name (if applicable)

March 18, 2026

Abstract

This thesis studies protein–protein binding affinity prediction from structure and asks one focused question: can short explicit-solvent molecular dynamics improve a residue-graph affinity model beyond a strong static baseline? The pipeline used in this study starts from a curated wild-type subset of PPB-Affinity, runs short molecular-dynamics simulations for each complex, constructs residue graphs with DeepRank2, adds molecular-dynamics-derived node and edge summaries, and trains either a masked graph variational autoencoder or a direct supervised graph regressor.

The final evaluation uses a matched 1437-complex subset for which both full-complex and interface-only graphs are available. The final model matrix contains 52 configurations evaluated under repeated outer resampling with 5 folds, 2 repeats, and 520 total fits. Two graph views are compared, full and interface, together with two feature modes, static (S) and static-plus-dynamic (SD).

The main results are clear. The full graph outperforms the interface graph in all 24 matched variational-model comparisons. The dynamic augmentation does not provide a robust held-out improvement over the static representation. The strongest overall model is the direct supervised baseline on the full static graph, with mean test RMSE 2.3379, Pearson correlation 0.5064, and R^2 0.2146. Semi-supervised dynamic models fit the training data more strongly but show larger train–test gaps.

The conclusion is therefore narrow but firm. In this pipeline, the molecular-dynamics-derived encoding used here does not improve affinity prediction over a well-constructed static residue graph. The result identifies the main directions for further work: richer dynamic representations, longer simulations, and less restrictive interface definitions.

Acknowledgments

I am grateful to my supervisor for guidance throughout the project and for repeated feedback on both the technical work and the written thesis. I also thank the staff of the School of Mathematics and Statistics and the collaborating research environment that made the computational work possible. Finally, I thank my family and friends for their support during the project.

Contents

1 Introduction **1**

1.1 Problem and motivation 1

1.2 Research questions and scope 1

1.3 What the thesis contributes 2

1.4 Thesis structure 2

2 Literature Review **3**

2.1 Protein–protein affinity prediction 3

2.2 Physics-based and empirical approaches 3

2.3 Why the protein–ligand comparison matters 4

2.4 Why use a variational graph model 4

2.5 Where this thesis fits 4

3 Background and Theoretical Context **5**

3.1 Affinity as the target quantity 5

3.2 Molecular dynamics as a source of descriptors 5

3.3 Residue graphs and message passing 6

3.4 Variational graph representation learning 6

3.5 Why repeated held-out evaluation matters 7

4 Methods **8**

4.1 Dataset curation and affinity labels 8

4.2 Molecular dynamics system preparation 9

4.3 Molecular dynamics protocol 9

4.4 Residue-graph construction 10

4.5 Molecular dynamics-derived node and edge features 10

4.6 Approximate inter-chain force features 11

4.7 Graph views and matched analysis subset 11

4.8 Feature modes and preprocessing 12

4.9 Masked graph variational autoencoder 12

4.10 Direct supervised baseline 13

4.11 Latent-space regression and final VAE evaluation . . . 14
4.12 Repeated outer resampling and model matrix . . . 14
4.13 Evaluation metrics . . . 15

5 Results **16**
5.1 Headline model performance . . . 16
5.2 Effect of graph view: full versus interface . . . 17
5.3 Effect of dynamic features: S versus SD . . . 18
5.4 Effect of supervision . . . 18
5.5 Effect of reconstruction target policy . . . 19
5.6 Overfitting and train–validation–test behaviour . . . 19
5.7 Summary of empirical conclusions . . . 21

6 Discussion and Conclusions **22**
6.1 Main findings . . . 22
6.2 Interpreting the full-versus-interface result . . . 22
6.3 Interpreting the negative dynamic result . . . 23
6.4 Why the supervised baseline won . . . 23
6.5 What the train–test gaps show . . . 23
6.6 Limitations . . . 23
6.7 Conclusion . . . 24

A Supplementary Technical Background **25**
A.1 Molecular dynamics in plain terms . . . 25
A.2 Residue graphs and graph neural networks in plain terms . . . 26
A.3 Variational autoencoders in plain terms . . . 26
A.4 Why use ridge regression on the latent space . . . 27
A.5 Why repeated outer resampling matters in practical terms . . . 27

B Archived Approaches and Project Evolution **29**
B.1 Correction learning around fast affinity baselines . . . 29
B.1.1 PRODIGY . . . 29
B.1.2 MM/GBSA . . . 30
B.2 The Cα contact-map branch . . . 30
B.3 The PPB-inspired augmentation and spatiotemporal branch . . . 31
B.4 Exploratory dataset curation utilities . . . 31
B.5 What the archived branches contributed . . . 31

C Complete Model Matrix **33**

D Repository and Implementation Availability **36**

List of Figures

List of Tables

5.1 Selected headline models from the final 52-configuration evaluation matrix. Metrics are means over 10 outer held-out fits on the matched 1437-complex set. For GraphVAE models, the reported predictive metrics come from the downstream ridge probe on latent means rather than solely from the neural affinity head. 17

5.2 Family-level train/validation/test behaviour. Each row is the arithmetic mean over all configurations in that family, using the configuration-level means from the 10 outer held-out fits. Here $g_r = \bar{r}_{\text{train}} - \bar{r}_{\text{test}}$. 20

C.1 Complete 52-model evaluation matrix, sorted by mean test RMSE. Metrics are means over 10 outer held-out fits. 34

Chapter 1

Introduction

1.1 Problem and motivation

Protein–protein interactions are biologically central and therapeutically important [1]. For many applications, the relevant question is not only whether two proteins bind, but how strongly they bind. The natural quantitative target is the binding free energy, or equivalently the dissociation constant K_D. Predicting this quantity from structure is appealing because it is experimentally measurable and relevant to biomolecular design [2, 3]. It is also difficult. A deposited structure is only one conformation, while the measured affinity reflects a thermodynamic average over many molecular states [2, 3].

This difficulty motivates the central question of the thesis: can short explicit-solvent molecular-dynamics simulations provide useful information beyond a carefully constructed static residue graph? The question is narrow on purpose. The thesis does not attempt rigorous free-energy calculation. It asks whether a practical MD-to-graph pipeline improves held-out prediction on a modern protein–protein affinity dataset.

1.2 Research questions and scope

The work is centred on one pipeline. Starting from a curated broad subset of PPB-Affinity, each complex is prepared for molecular dynamics, simulated in explicit solvent, converted to a residue graph with DeepRank2, augmented with trajectory-derived descriptors, and then passed to either a masked GraphVAE or a direct supervised baseline. The final evaluation is restricted to a matched 1437-complex subset for which both full and interface graph views can be materialized successfully.

Within that scope, the thesis asks four concrete questions. First, is a full residue graph more predictive than an interface-only graph? Second, do molecular-dynamics-derived features improve prediction relative to static graph features alone? Third, does a masked GraphVAE provide a better representation than a direct supervised graph regressor when

both models receive the same inputs? Fourth, what do repeated held-out evaluations reveal about underfitting, overfitting, and the reliability of conclusions drawn from a single split?

1.3 What the thesis contributes

The main contribution is a fully specified and reproducible comparison. The thesis evaluates full versus interface graphs, static versus static-plus-dynamic features, GraphVAE versus direct supervised regression, and unsupervised versus semi-supervised variational training under repeated outer resampling rather than a single favourable split. The goal is not to claim that dynamics must help. The goal is to determine, as cleanly as possible, what the implemented pipeline actually supports.

1.4 Thesis structure

Chapter 2 places the work in the context of protein–protein affinity prediction, molecular simulation, and learned structural representations. Chapter 3 introduces the technical ideas needed for the rest of the thesis. Chapter 4 describes the implemented pipeline in replication-level detail. Chapter 5 reports the final model matrix. Chapter 6 interprets the results, states the main limitations, and gives the final conclusions. The appendices provide additional technical background, archived project branches, and a map from the written thesis to the public repository.

Chapter 2

Literature Review

2.1 Protein–protein affinity prediction

Protein–protein affinity prediction remains less standardized than several neighbouring tasks in structural bioinformatics [2, 3]. Much of the older supervised literature focused on mutation-induced changes in binding energy, supported by resources such as SKEMPI 2.0, rather than on direct prediction of absolute binding affinity for heterogeneous wild-type complexes [4]. Absolute-affinity benchmarks do exist, including the structure-based benchmark assembled by Kastritis and coworkers, but they are small by current deep-learning standards [5]. PPB-Affinity is therefore an important recent development because it was assembled specifically for direct protein–protein affinity prediction at larger scale [3].

This task is difficult for straightforward physical reasons. The target is thermodynamic, but the input is usually a single experimentally observed structure. Reviews of the field emphasize that limited labelled data, structural heterogeneity, and the gap between one structure and an underlying binding ensemble remain major obstacles [2, 3].

2.2 Physics-based and empirical approaches

One response is to estimate affinity directly from simulation. Umbrella sampling and alchemical free-energy methods are established routes to free-energy estimation, but they are computationally expensive and sensitive to protocol design [6, 7]. At lower cost, MM/PBSA and MM/GBSA provide widely used endpoint approximations, although their performance depends strongly on solvent treatment, entropy approximations, and state averaging [8]. This literature motivates taking molecular dynamics seriously, but it does not make full free-energy calculation the only practical option.

Another response is to predict affinity directly from structure. PRODIGY is a representative example of a fast empirical model built from interfacial contact descriptors [9].

Other earlier approaches similarly relied on engineered structural features and conventional regressors [10]. More recent work has moved toward learned structural representations. The PPB-Affinity paper includes a benchmark deep model on the new dataset [3], and recent studies such as PPI-Graphomer have explored pretrained and graph-based models for direct affinity prediction [11].

2.3 Why the protein–ligand comparison matters

The protein–ligand literature provides a useful contrast. Datasets such as PDBbind have given that field a longer and more standardized benchmark tradition [12]. This has supported a broad range of learned structural models, including voxelized approaches such as Pafnucy [13] and more recent trajectory-aware models trained on MD-derived data [14]. The protein–ligand literature therefore provides concrete examples of successful geometric and trajectory-aware affinity models.

Protein–protein complexes make the representation question less obvious. A ligand naturally defines a local binding object inside a pocket; a protein–protein complex does not. Even before model choice, one must decide whether the relevant representation is the whole complex, the interface only, or some local patch around the interface. That is a central design question in this thesis.

2.4 Why use a variational graph model

Variational autoencoders are well established in molecular representation learning as a way to combine reconstruction with latent-space regularization [15–18]. They are not the default standard in direct protein–protein affinity prediction, where supervised regression remains more common [2, 11]. They are still relevant here because the thesis asks not only which predictor is strongest, but also whether a reconstruction-based latent representation can capture useful structure in residue graphs enriched with molecular-dynamics-derived features.

2.5 Where this thesis fits

The literature suggests a clear gap. Rigorous free-energy methods are principled but expensive. Fast empirical scorers are cheap but compressed. Learned structural models are becoming more capable, but direct protein–protein affinity prediction still lacks a single settled representation or modelling strategy. This thesis therefore asks a focused question: starting from a residue-graph representation of a protein complex, do short explicit-solvent molecular-dynamics summaries improve held-out affinity prediction, and does that answer depend on graph view or model family?

Chapter 3

Background and Theoretical Context

3.1 Affinity as the target quantity

The quantitative target in this thesis is the standard Gibbs free energy of binding, ΔG. For a complex written schematically as

$$\mathrm{P}_1 + \mathrm{P}_2 \rightleftharpoons \mathrm{P}_1\mathrm{P}_2, \tag{3.1}$$

the equilibrium dissociation constant is

$$K_D = \frac{[\mathrm{P}_1][\mathrm{P}_2]}{[\mathrm{P}_1\mathrm{P}_2]}, \tag{3.2}$$

and the corresponding free energy is

$$\Delta G = RT \ln K_D. \tag{3.3}$$

This matters because it gives a single continuous regression target, but it also highlights the central difficulty of the problem: affinity is a thermodynamic quantity, whereas the model sees only structure-derived features from a finite set of observed conformations [3].

3.2 Molecular dynamics as a source of descriptors

In classical molecular dynamics, atomic coordinates evolve under Newton's equations of motion,

$$m_i \frac{d^2\mathbf{r}_i}{dt^2} = -\nabla_{\mathbf{r}_i} U(\mathbf{r}_1, \ldots, \mathbf{r}_N), \tag{3.4}$$

where U is the force-field potential energy. In this thesis, molecular dynamics is used descriptively rather than as a direct free-energy method. The goal is to extract trajectory-derived summaries such as hydration, flexibility, and contact persistence and test whether

they improve a downstream affinity predictor. This is plausible because proteins are dynamic rather than rigid objects, but the usefulness of such descriptors depends on simulation length, feature design, and model integration [19].

3.3 Residue graphs and message passing

A protein complex can be represented as a graph $G = (V, E)$ in which residues form the nodes and selected residue pairs form the edges. This representation is a natural compromise between atom-level detail and computational tractability, and it aligns well with residue-level descriptors such as solvent exposure, buried surface area, residue depth, and physicochemical identity [20]. In the present thesis, the graph topology is static, while some node and edge attributes summarize time-dependent behaviour from molecular dynamics.

Graph neural networks learn on such inputs by message passing. In a generic layer,

$$\mathbf{h}_i^{(\ell+1)} = \phi^{(\ell)} \left(\mathbf{h}_i^{(\ell)}, \sum_{j \in \mathcal{N}(i)} \psi^{(\ell)} \left(\mathbf{h}_i^{(\ell)}, \mathbf{h}_j^{(\ell)}, \mathbf{e}_{ij} \right) \right), \tag{3.5}$$

where $\mathbf{h}_i^{(\ell)}$ is the node state, $\mathbf{e}_{ij}$ is the edge feature vector, and $\psi^{(\ell)}$ and $\phi^{(\ell)}$ are learnable message and update functions. After several layers, node states are pooled into a graph embedding for downstream prediction. The models used in this thesis employ GINE-style message passing, which is edge-aware and therefore appropriate when both static and dynamic edge features are present [21, 22].

3.4 Variational graph representation learning

The GraphVAE used in this thesis is a latent-variable model that combines graph encoding, masked reconstruction, and latent regularization. In the standard VAE formulation, training optimizes a variational lower bound,

$$\log p_\theta(\mathbf{x}) \geq \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \left[\log p_\theta(\mathbf{x} \mid \mathbf{z}) \right] - D_{\mathrm{KL}}(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, p(\mathbf{z})), \tag{3.6}$$

where the reconstruction term encourages the latent variable to retain useful information about the input and the Kullback–Leibler term keeps the latent distribution close to a simple prior [15]. In this thesis, the model is used as a representation learner rather than as a full generative model of protein graphs. The encoder maps a residue graph to a graph-level latent distribution, and the decoder reconstructs selected masked node and edge features. A supervised affinity head can optionally be attached to the latent mean.

3.5 Why repeated held-out evaluation matters

When many closely related models are compared, a single train/validation/test split can give a misleading impression of relative performance [23]. For that reason, the final conclusions in this thesis are based on repeated outer held-out evaluation rather than on a one-off split. The point of the final evaluation matrix is not only to identify the strongest configuration, but to compare design choices such as full versus interface graphs, static versus dynamic features, and variational versus direct supervised training under a more stable evaluation protocol.

Chapter 4

Methods

4.1 Dataset curation and affinity labels

All experiments were based on the PPB-Affinity resource, a large protein–protein binding affinity dataset assembled for machine-learning applications in macromolecular drug discovery [3]. The dataset used in this study was a locally curated broad subset that retained all major PPB-Affinity subgroups while enforcing a single annotated ligand–receptor pairing per complex. Complexes were retained only if a Protein Data Bank (PDB) structure, an interpretable ligand and receptor chain assignment, and an affinity measurement sufficient to derive a binding free energy were all available. Mutant complexes were excluded so that the final learning problem remained a wild-type structure-to-affinity mapping. Duplicate entries were removed by requiring both a unique PDB identifier and a unique ordered ligand/receptor chain-group definition. The resulting curated CSV contained 2818 complexes.

Each complex was assigned a canonical identifier of the form `PDB__ligandChains__receptorChain` Experimental affinity was represented as the standard Gibbs free energy of binding, ΔG, in kcal mol^{-1}. When a free energy value was not already present in the source table, it was computed from the reported dissociation constant K_D using

$$\Delta G = RT \ln K_D, \tag{4.1}$$

where $R = 0.00198720425864083$ kcal mol^{-1} K^{-1} and T is the experimental temperature in kelvin. If a complex-specific temperature was not available, $T = 298.15$ K was used. This conversion was applied uniformly during dataset normalization so that all downstream models were trained against a single continuous thermodynamic target.

4.2 Molecular dynamics system preparation

Protein complexes were prepared from their PDB coordinate files using PDBFixer [24] and the OpenMM application layer [25]. The annotated ligand and receptor chains were first extracted from the source complex. Missing residues were detected during preprocessing but were not rebuilt; this choice preserved the experimentally deposited backbone definition rather than introducing modeled insertions. Nonstandard residues were converted when PDBFixer provided a standard mapping, heterogens were removed, missing heavy atoms were added, and hydrogens were assigned before solvation. Only protein residues were retained in the final simulation system prior to solvent addition.

Prepared complexes were solvated explicitly with 1.0 nm of solvent padding around the solute and with an ionic strength of 0.15 M. The force-field stack used throughout this study was `amber14-all.xml` for the protein and `amber14/tip3pfb.xml` for the solvent, corresponding to the Amber14 protein force field and the TIP3P-FB water model [26, 27]. Protonation-state preparation used the per-complex experimental pH when available and otherwise defaulted to pH 7.0. In the final retained analysis set, experimental pH metadata were not available and the default pH was therefore used uniformly.

4.3 Molecular dynamics protocol

All simulations were carried out in OpenMM [25]. The molecular mechanics system was built with particle-mesh Ewald electrostatics, a 1.0 nm real-space nonbonded cutoff, constraints on bonds involving hydrogen, rigid water, and an Ewald error tolerance of 10^{-4}. Pressure was controlled at 1 atm using a Monte Carlo barostat and temperature was controlled with a Langevin middle integrator using a friction coefficient of 1 ps^{-1} and a 2 fs time step. Each system was energy-minimized for up to 1000 iterations, equilibrated for 100,000 steps, and then propagated for a 500,000-step production run. At a 2 fs step size, these settings correspond to 0.2 ns of equilibration followed by 1.0 ns of additional simulation.

Trajectory and state reporters were written every 5000 integration steps, corresponding to 10 ps between saved frames. Two NetCDF trajectories were written for each complex: a full-system trajectory containing protein, solvent, and ions, and a second protein-only trajectory written in the same coordinate frame. Because the reporters were attached before equilibration began, all saved frames from both the equilibration and production phases were available to downstream feature extraction. The stored outputs for each complex therefore comprised 120 saved frames over a total simulated duration of 1.2 ns. Complex-specific temperatures from the source dataset were used when available; otherwise a default temperature of 298.15 K was used.

4.4 Residue-graph construction

Residue-level graphs were generated with DeepRank2 [20]. To avoid restricting the graph to a predefined interface only, the preparation pipeline used DeepRank2 in a full-complex query mode with an effectively unbounded influence radius of 10^6 Å, which ensured that all protein residues in the prepared topology were represented in the resulting graph. DeepRank2 feature modules `components`, `contact`, `exposure`, and `surfacearea` were applied to the protein-only topology written from the molecular dynamics preparation stage. This yielded per-residue structural descriptors including buried surface area, solvent-accessible surface area, residue depth, and half-sphere exposure features, together with pairwise edge descriptors including residue distance, same-chain identity, electrostatic interaction, and van der Waals interaction.

These DeepRank2 features were augmented with two additional categories of node attributes. First, residue identity was encoded using a 20-way amino-acid one-hot representation together with coarse physicochemical indicators and scalar descriptors of charge, hydropathy, mass, aromaticity, and hydrogen-bonding tendency. Second, a local structural-context count was computed for each residue on the first saved protein frame by counting residues within 8 Å on the same chain and on the partner chain. These context counts were concatenated with the DeepRank2 node features and treated as static inputs in all downstream models.

4.5 Molecular dynamics-derived node and edge features

Dynamic graph attributes were computed from the saved trajectories using MDTraj [28]. For each complex, at most 120 frames were used during feature extraction; given the reporting schedule above, this corresponded to using all saved frames from the 1.2 ns trajectory. Dynamic node features comprised hydration, flexibility, backbone torsional state, and inter-chain force-proxy summaries. Dynamic edge features comprised contact frequency and distance statistics evaluated on the fixed edge set of the static graph.

Hydration was quantified from the full solvated trajectory. For each residue and frame, all unique water oxygen residues within 5 Å of any heavy atom of that residue were counted. The stored features were $\log(1 + \bar{w}_i)$, where $\bar{w}_i$ is the mean water count for residue i across frames, and the standard deviation of the framewise water counts. The logarithm compresses the right tail produced by highly solvent-exposed residues while preserving the distinction between low-count environments near zero. Conformational flexibility was represented by the Cα root-mean-square fluctuation (RMSF) computed after least-squares superposition of the trajectory onto itself using Cα atoms. Backbone torsional state was summarized by computing ϕ and ψ angles for all residues with defined

dihedrals and storing the frame-averaged values of $\sin\phi$, $\cos\phi$, $\sin\psi$, and $\cos\psi$.

Dynamic edge features were computed only for residue pairs connected by an existing static graph edge. For each such pair, MDTraj's closest-heavy-atom distance was evaluated in every saved frame. The dynamic contact frequency for edge (i, j) was then defined as the fraction of frames in which the closest-heavy distance was at most 8 Å. When distance statistics were enabled, the mean and standard deviation of the closest-heavy distance across frames were also stored as dynamic edge attributes. This representation should therefore be interpreted as a static graph with time-aggregated edge annotations rather than as a fully time-varying graph.

4.6 Approximate inter-chain force features

In addition to hydration and conformational descriptors, each residue was assigned an approximate inter-chain nonbonded force summary. These features were derived from the protein-only trajectory and a reconstructed OpenMM analysis system built from the saved protein topology using the same force-field files as the original simulation. To isolate direct ligand–receptor interactions, the original OpenMM `NonbondedForce` parameters were copied into two separate `CustomNonbondedForce` objects, one representing a Coulomb term and one representing a Lennard–Jones term, and each custom force was restricted to ligand-atom versus receptor-atom interaction groups using OpenMM interaction-group semantics [25]. The custom analysis system used periodic cutoffs and excluded the original exception pairs; long-range correction was disabled for the custom forces.

For each saved frame, per-atom forces from the Coulomb and Lennard–Jones custom terms were accumulated over all atoms belonging to the same residue. The Euclidean norm of the resulting residue-level force vector was then taken, and the mean and standard deviation of these norms across frames were stored separately for the Coulomb and Lennard–Jones channels. Because these quantities were computed on reconstructed protein-only systems with direct ligand–receptor interaction groups rather than extracted directly from the full solvated PME simulation context, they should be interpreted as approximate inter-chain nonbonded force proxies rather than exact total molecular dynamics forces.

4.7 Graph views and matched analysis subset

Two graph views were evaluated. The *full* view retained the complete residue graph for each complex. The *interface* view retained only a fixed-size local patch intended to emphasize the interaction region while keeping the input size approximately constant across complexes. In the final evaluation, the interface policy was `md_closest_pair_patch`. Un-

der this policy, mapped Cα coordinates from the molecular dynamics trajectory were used to identify the ligand residue and receptor residue with the minimum inter-chain Cα distance. These two residues formed the interface core, and the final node set was obtained by expanding locally around this core under a hard cap of 128 residues. The corresponding edge set was induced from the original graph.

To support a fair comparison between the full and interface representations, both views were restricted to the same retained complex subset. Specifically, the interface materialization stage was run in a matched-subset mode so that any complex without a valid interface patch was removed from both views. The final evaluation therefore used 1437 matched complexes, each represented once as a full graph and once as an interface graph derived from the same prepared record.

4.8 Feature modes and preprocessing

Two input modes were considered. In the *static* mode, denoted S, model inputs consisted only of the static node and edge features described above. In the *static-plus-dynamic* mode, denoted SD, the input feature vector was augmented with the dynamic node features, torsion summaries, force-proxy summaries, and dynamic edge features. The edge feature `same_chain` was retained as an input feature in both modes but was excluded from reconstruction targets because it is topologically trivial once the graph has been constructed.

Feature scaling was fit exclusively on the training split of each run. For every numeric feature, the training distribution was first clipped to its 0.5th and 99.5th percentiles, then standardized to zero mean and unit variance, and finally clipped again to the interval $[-8, 8]$. The same transformation was then applied unchanged to the corresponding validation and test splits. This preprocessing was identical for the GraphVAE and the direct supervised baseline, ensuring that any performance differences were attributable to model design rather than to differences in feature normalization.

4.9 Masked graph variational autoencoder

The unsupervised and semi-supervised graph representation models were implemented as masked graph variational autoencoders using PyTorch Geometric [29]. Both node and edge inputs were projected linearly to a hidden dimension of 128 and passed through three graph isomorphism network with edge features (GINE) convolution layers [22]. Graph-level embeddings were obtained by concatenating global mean pooling and global max pooling. Two linear heads then produced the latent mean vector μ and log-variance vector $\log \sigma^2$. The latent dimensionality was varied over $d_z \in \{8, 16, 32\}$ in the final model matrix.

The decoder reconstructed selected feature subsets rather than the full input tensor. For each batch, target node and edge entries were independently masked with probability 0.30. Decoder inputs then consisted of the sampled latent vector together with the unmasked graph inputs, the partially masked target features, and explicit binary mask indicators. Three target policies were used in the final evaluation. The `shared_static` policy reconstructed static residue descriptors and the nontrivial static edge distance feature. The `sd_dynamic_all` policy reconstructed only dynamic node and dynamic edge attributes. The `sd_static_plus_dynamic_all` policy reconstructed the union of static and dynamic targets. Only the `shared_static` policy was used for S runs, whereas all three policies were used for SD runs.

For a batch of graphs, the total GraphVAE objective was

$$\mathcal{L} = \mathcal{L}_{\text{node}}^{\text{recon}} + \mathcal{L}_{\text{edge}}^{\text{recon}} + \beta \mathcal{L}_{\text{KL}} + \lambda_{\text{corr}} \mathcal{L}_{\text{corr}} + \lambda_{\text{aff}} \mathcal{L}_{\text{aff}}, \tag{4.2}$$

where the node and edge reconstruction terms were masked mean-squared errors over the selected target entries, the Kullback–Leibler term was

$$\mathcal{L}_{\text{KL}} = \frac{1}{B d_z} \sum_{b=1}^{B} \sum_{k=1}^{d_z} \left[-\frac{1}{2} \left(1 + \log \sigma_{bk}^2 - \mu_{bk}^2 - \sigma_{bk}^2 \right) \right], \tag{4.3}$$

and $\mathcal{L}_{\text{corr}}$ penalized the squared off-diagonal elements of the latent covariance matrix to discourage redundant latent dimensions. In semi-supervised runs, $\mathcal{L}_{\text{aff}}$ was a mean-squared error between the normalized affinity prediction from a latent MLP head and the normalized experimental target. The final settings were $\lambda_{\text{corr}} = 0.01$, $\lambda_{\text{aff}} = 1.0$ for semi-supervised runs and 0 otherwise, $\beta_{\text{start}} = 0$, $\beta_{\text{end}} = 1$, and a linear β warm-up over the first 30% of training epochs. With a maximum of 120 epochs, the KL weight therefore increased linearly over the first 36 epochs and remained at 1.0 thereafter.

All GraphVAE models were optimized with AdamW using a learning rate of 10^{-4}, weight decay of 10^{-5}, batch size 16, and gradient clipping at a global norm of 5. Training used a maximum of 120 epochs with early stopping patience of 20 epochs. Checkpoint selection was based on the validation reconstruction loss, plus the weighted validation affinity loss in semi-supervised runs. The KL and correlation terms regularized training but did not directly determine which checkpoint was retained as the best model.

4.10 Direct supervised baseline

To separate the value of the graph representation from the value of the variational objective, each graph view and feature mode was also evaluated with a direct supervised baseline. This model used the same input feature definitions, the same hidden dimension of 128, the same three GINE layers, and the same mean-plus-max graph pooling strategy

as the GraphVAE encoder. The difference was that the pooled graph embedding was passed directly to a multilayer perceptron regressor trained to predict ΔG with a mean-squared-error loss. The supervised baseline was optimized with AdamW using the same learning rate, weight decay, batch size, and maximum epoch count as the GraphVAE runs. Early stopping was based on validation mean squared error, with patience fixed to 20 epochs in the final evaluation matrix.

4.11 Latent-space regression and final VAE evaluation

Final GraphVAE performance was not reported solely from the neural affinity head. Instead, each retained checkpoint was treated as a representation learner, and its graph-level latent mean vectors were exported for the train, validation, and test splits. A ridge regressor was then fit on the training latent vectors only,

$$\hat{\mathbf{w}} = \arg\min_{\mathbf{w}} \|\mathbf{y}_{\text{train}} - \mathbf{X}_{\text{train}}\mathbf{w}\|_2^2 + \alpha\|\mathbf{w}\|_2^2, \tag{4.4}$$

and evaluated on the held-out validation and test splits. The regularization parameter was selected with `RidgeCV` from scikit-learn [30] over the grid

$$\alpha \in \{10^{-4}, 3\times10^{-4}, 10^{-3}, 3\times10^{-3}, 10^{-2}, 3\times10^{-2}, 10^{-1}, 3\times10^{-1}, 1, 3, 10, 30, 100\},$$

using five-fold inner cross-validation on the training split only. No additional repeated cross-validation was performed inside this probe during the final resampling study. The headline GraphVAE metrics reported in the results chapter therefore quantify the predictive quality of the learned latent representation under a simple regularized linear readout, rather than the performance of the end-to-end neural affinity head alone.

4.12 Repeated outer resampling and model matrix

The final evaluation used repeated outer resampling on the matched 1437-complex set. Unique complex identifiers, rather than individual graphs or trajectory frames, were the unit of resampling. For each of two repeats, complexes were randomly permuted with seed 2026 and partitioned into five outer folds. One fold was held out as the test set and the remaining four folds were split into training and validation subsets using a validation fraction of 0.1875 of the train-plus-validation pool, which yields the intended overall 70/15/15 proportion within each outer fit. In practice, each fit therefore contained approximately 934 training complexes, 215–216 validation complexes, and 287–288 test complexes. All preprocessing, model fitting, checkpoint selection, and ridge fitting were

repeated independently within each outer split.

The final model matrix contained 52 configurations. Forty-eight of these were Graph-VAE runs spanning two graph views (full and interface), two input modes (S and SD), two supervision settings (unsupervised and semi-supervised), and three latent dimensionalities (8, 16, and 32). For SD runs, three target policies were evaluated; for S runs, only `shared_static` was evaluated because the dynamic target policies are not distinct when dynamic inputs are absent. Four additional runs were direct supervised baselines corresponding to the Cartesian product of graph view and feature mode. All models used the same hidden dimension (128), number of GINE layers (3), learning rate (10^{-4}), weight decay (10^{-5}), batch size (16), maximum epochs (120), and early-stopping patience (20) in the final evaluation matrix.

4.13 Evaluation metrics

For every outer fit, predictive performance was summarized on the held-out test set using root-mean-square error (RMSE), coefficient of determination (R^2), and Pearson correlation coefficient. For n test complexes with observed affinities y_i and predictions $\hat{y}_i$, these were computed as

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}, \tag{4.5}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}, \tag{4.6}$$

and

$$r = \frac{\sum_{i=1}^{n}(y_i - \bar{y})(\hat{y}_i - \bar{\hat{y}})}{\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}\sqrt{\sum_{i=1}^{n}(\hat{y}_i - \bar{\hat{y}})^2}}. \tag{4.7}$$

For the final report, metrics were aggregated across the 10 outer fits per configuration and summarized primarily by their mean. This repeated held-out evaluation formed the basis of all factor-level conclusions about graph view, feature mode, supervision, target policy, and latent dimensionality.

Chapter 5

Results

This chapter reports the final evaluation described in Chapter 4. The analysis was restricted to the matched 1437-complex subset for which both the full and interface graph views were available after molecular-dynamics processing, force-feature construction, and interface materialization. The final model matrix comprised 52 configurations: 48 GraphVAE-based representation models and 4 direct supervised baselines. Each configuration was evaluated across 10 outer held-out fits obtained from 5 folds and 2 repeats. If $m_{c,t}$ denotes a metric for configuration c on outer fit $t \in \{1, \ldots, 10\}$, the reported configuration-level summary is

$$\bar{m}_c = \frac{1}{10} \sum_{t=1}^{10} m_{c,t}. \tag{5.1}$$

All rankings and factor comparisons below are based on these configuration-level means. For matched design comparisons, the chapter reports the mean paired effect size together with the number of pairings favoring one design over the other. The complete 52-model table is given in Appendix C.

5.1 Headline model performance

Table 5.1 lists the strongest configurations from the final matrix. The best overall model was the direct supervised baseline on the full static graph, `full_base_S`, with mean test RMSE 2.3379, mean test Pearson correlation 0.5064, and mean test R^2 0.2146. The best GraphVAE configuration was `full_S_semi_shared_static_z32`, with mean test RMSE 2.4292, mean test Pearson correlation 0.4649, and mean test R^2 0.1537. The best static-plus-dynamic GraphVAE configuration was `full_SD_semi_shared_static_z8`, with mean test RMSE 2.4364 and mean test Pearson correlation 0.4749. The strongest interface baseline, `iface_base_S`, achieved mean test RMSE 2.5782, while the best interface GraphVAE, `iface_S_semi_shared_static_z8`, achieved mean test RMSE 2.6041.

The best direct supervised baseline outperformed the best GraphVAE by 0.0913 RMSE

Configuration	Test RMSE	Test r	Test R^2	Test−Train RMSE gap
`full_base_S`	2.3379	0.5064	0.2146	0.8021
`full_base_SD`	2.4073	0.4802	0.1685	0.8414
`full_S_semi_shared_static_z32`	2.4292	0.4649	0.1537	0.7451
`full_SD_semi_shared_static_z8`	2.4364	0.4749	0.1478	1.1303
`iface_base_S`	2.5782	0.3451	0.0470	0.8090
`iface_S_semi_shared_static_z8`	2.6041	0.3402	0.0282	0.8018
`iface_SD_semi_sd_static_plus_dynamic_z32`	2.6238	0.3252	0.0146	0.7597

Table 5.1: Selected headline models from the final 52-configuration evaluation matrix. Metrics are means over 10 outer held-out fits on the matched 1437-complex set. For GraphVAE models, the reported predictive metrics come from the downstream ridge probe on latent means rather than solely from the neural affinity head.

units and by 0.0415 Pearson-correlation units on average. Within the SD family, the best model was again a full-graph method rather than an interface method: `full_base_SD` reached mean test RMSE 2.4073 and mean test $r = 0.4802$, while the best SD GraphVAE reached mean test RMSE 2.4364 and mean test $r = 0.4749$. The final evaluation therefore did not support a claim that the variational objective outperformed a direct supervised graph regressor on the present dataset and feature set.

5.2 Effect of graph view: full versus interface

The largest factor effect in the matrix was the advantage of the full graph view over the interface graph view. There were 24 matched GraphVAE configuration pairs differing only in whether the model was trained on `full` or `interface` records. In all 24 of these paired comparisons, the full graph achieved a lower mean test RMSE than the corresponding interface graph. Defining the paired RMSE penalty for the interface representation as

$$\Delta_{\mathrm{RMSE}}^{\mathrm{iface-full}} = \mathrm{R\bar{M}SE}_{\mathrm{iface}} - \mathrm{R\bar{M}SE}_{\mathrm{full}}, \tag{5.2}$$

the mean paired penalty was 0.1005 RMSE units. If the corresponding Pearson-correlation advantage is defined as

$$\Delta_{r}^{\mathrm{full-iface}} = \bar{r}_{\mathrm{full}} - \bar{r}_{\mathrm{iface}}, \tag{5.3}$$

the mean full-graph advantage was 0.0993.

The same direction held at the family level. Averaging over all 24 full-graph GraphVAE configurations gave mean test RMSE 2.5494 and mean test Pearson correlation 0.2883. Averaging over all 24 interface GraphVAE configurations gave mean test RMSE 2.6498 and mean test Pearson correlation 0.1890. The direct supervised baselines showed the same pattern. The best full baseline, `full_base_S`, achieved mean test RMSE 2.3379, whereas the best interface baseline, `iface_base_S`, achieved mean test RMSE 2.5782. This confirms that the superiority of the full representation was not specific to the variational

model family.

This pattern was uniform enough that the main view-level conclusion is not sensitive to model-selection details. On the matched subset used here, with the residue-graph definitions and interface-patch construction described in Chapter 4, the full-graph representation was consistently more predictive than the interface-only representation.

5.3 Effect of dynamic features: S versus SD

The second main question was whether augmenting the static graph with molecular-dynamics-derived features improved predictive performance. The cleanest comparison for this purpose is between matched S and SD configurations under the `shared_static` reconstruction target, because both models are then trained under the same target policy and differ only in whether the dynamic channels are included as inputs. There were 12 such matched GraphVAE pairs in the final matrix. In 11 of the 12 pairs, the S model achieved a lower mean test RMSE than the corresponding SD model.

Defining the paired dynamic-feature penalty as

$$\Delta_{\mathrm{RMSE}}^{\mathrm{SD-S}} = \mathrm{R\bar{M}SE}_{\mathrm{SD}} - \mathrm{R\bar{M}SE}_{\mathrm{S}}, \tag{5.4}$$

the mean penalty was 0.0254 RMSE units. The corresponding Pearson-correlation difference,

$$\Delta_{r}^{\mathrm{S-SD}} = \bar{r}_{\mathrm{S}} - \bar{r}_{\mathrm{SD}}, \tag{5.5}$$

favored the static-only representation by 0.0215 on average. The direct supervised baselines were consistent with the same conclusion: `full_base_SD` was worse than `full_base_S` by 0.0694 RMSE units, and `iface_base_SD` was not better than `iface_base_S`.

This effect was modest. The best SD GraphVAE, `full_SD_semi_shared_static_z8`, was only 0.0072 RMSE units worse than the best S GraphVAE, `full_S_semi_shared_static_z32`, and it had a slightly higher test Pearson correlation (0.4749 versus 0.4649). The result is therefore not that dynamic features are uniformly harmful. It is that, across the full evaluation matrix, the dynamic augmentation does not yield a robust and repeatable generalization gain over the static representation.

5.4 Effect of supervision

The influence of semi-supervision depended strongly on graph view. Within the full graph view, semi-supervision helped consistently. For the static mode, all 3 of the 3 matched full-graph pairs favored semi-supervision over the corresponding unsupervised model, with a mean test-RMSE gain of 0.1640. For the static-plus-dynamic mode, all 9 of the 9 matched full-graph pairs favored semi-supervision, with a mean test-RMSE gain of 0.1604. The

strongest full-graph models therefore all came from the semi-supervised branch of the matrix.

The interface results were weaker and less coherent. In the static mode, semi-supervision still helped in all 3 of the 3 matched interface pairs, but the mean gain was only 0.0290 RMSE units. In the dynamic mode, semi-supervision did not produce a robust benefit. Across the 9 matched interface SD pairs, the mean semi-supervised gain was -0.0081 RMSE units and only 3 of the 9 pairs favored the semi-supervised model. The interpretation is therefore not simply that supervision helps universally. Rather, supervision helps when the graph representation contains sufficient predictive structure, which appears to be true for the full graphs and only marginally true for the interface graphs.

5.5 Effect of reconstruction target policy

Within the SD GraphVAE branch, the final evaluation also tested whether the decoder should reconstruct only static targets, only dynamic targets, or both. The most informative regime for this comparison is the full-graph semi-supervised SD family, because this was the only dynamic branch that approached competitive held-out performance. Averaging over the three latent dimensionalities gave mean test RMSE 2.4434 for `shared_static`, 2.4795 for `sd_static_plus_dynamic_all`, and 2.5150 for `sd_dynamic_all`. The corresponding mean test Pearson correlations were 0.4682, 0.4335, and 0.4221, respectively.

Pairwise comparisons by latent dimensionality showed that reconstructing only dynamic targets was consistently worse than reconstructing the shared static targets. Across the three matched latent-dimensionality pairs, the mean RMSE penalty of `sd_dynamic_all` relative to `shared_static` was 0.0716. The mean Pearson-correlation loss was 0.0461. The `sd_static_plus_dynamic_all` target policy was intermediate. Its mean RMSE penalty relative to `shared_static` was 0.0360, and the corresponding mean Pearson-correlation loss was 0.0347.

These results indicate that, in the only dynamic regime that was even moderately competitive, asking the decoder to reconstruct dynamic targets did not improve held-out predictive quality. The strongest dynamic models were obtained when the dynamic channels were present as inputs but the reconstruction objective remained focused on the shared static targets.

5.6 Overfitting and train–validation–test behaviour

The train–validation–test metrics also showed a consistent generalization pattern. To quantify overfitting, define the train–test RMSE gap and train–test correlation gap for configuration c as

$$g_{\mathrm{RMSE}}(c) = \bar{\mathrm{RMSE}}_{\mathrm{test},c} - \bar{\mathrm{RMSE}}_{\mathrm{train},c}, \tag{5.6}$$

Family	Train r	Val r	Test r	g_r	Train RMSE	Test RMSE
Full baseline S	0.8225	0.5337	0.5064	0.3161	1.5358	2.3379
Full baseline SD	0.8077	0.5191	0.4802	0.3275	1.5659	2.4073
Full semi-supervised VAE S	0.7494	0.4666	0.4551	0.2942	1.7385	2.4369
Full semi-supervised VAE SD	0.8218	0.4580	0.4413	0.3805	1.4744	2.4793
Interface semi-supervised VAE S	0.7083	0.3356	0.3361	0.3722	1.8467	2.6112
Interface semi-supervised VAE SD	0.7685	0.3231	0.3189	0.4496	1.6479	2.6620

Table 5.2: Family-level train/validation/test behaviour. Each row is the arithmetic mean over all configurations in that family, using the configuration-level means from the 10 outer held-out fits. Here $g_r = \bar{r}_{\text{train}} - \bar{r}_{\text{test}}$.

and

$$g_r(c) = \bar{r}_{\text{train},c} - \bar{r}_{\text{test},c}. \tag{5.7}$$

Large positive values of either quantity indicate stronger train-to-test divergence.

The unsupervised GraphVAE families were characterized primarily by underfitting rather than by severe overfitting. For example, the mean train, validation, and test Pearson correlations for the full unsupervised static family were 0.2486, 0.1897, and 0.2055, respectively, with a mean train–test RMSE gap of only 0.0276. The corresponding full unsupervised dynamic family had mean train, validation, and test correlations of 0.1809, 0.0967, and 0.1074. These small train–test gaps do not indicate strong robustness; they indicate that the unsupervised models did not learn much affinity signal even on their training sets.

The overfitting story emerges in the semi-supervised families. The full semi-supervised static family had mean train, validation, and test Pearson correlations of 0.7494, 0.4666, and 0.4551, respectively, with a mean train–test RMSE gap of 0.6984. This is a moderate and credible generalization gap. The full semi-supervised dynamic family fit the training data substantially more strongly, with mean train correlation 0.8218, but it did not improve held-out performance; its mean test correlation was only 0.4413 and its mean train–test RMSE gap increased to 1.0049. The interface semi-supervised dynamic family exhibited the strongest divergence of all major families, with mean train, validation, and test correlations of 0.7685, 0.3231, and 0.3189, respectively, and a mean train–test RMSE gap of 1.0140.

Table 5.2 summarizes these family-level patterns. The best direct supervised baseline, `full_base_S`, combined the highest held-out performance with a more controlled generalization gap than the semi-supervised dynamic families. Its mean train, validation, and test correlations were 0.8225, 0.5337, and 0.5064, respectively, and its mean train–test RMSE gap was 0.8021. This gap is smaller than those of the dynamic semi-supervised VAE families while producing better held-out accuracy.

The worst individual overfitting case in the semi-supervised GraphVAE matrix was `iface_SD_semi_shared_static_z32`. This model reached mean train Pearson correlation

0.8213 but only mean test Pearson correlation 0.3185, giving a train–test correlation gap of 0.5028. Its mean train RMSE was 1.4239, while its mean test RMSE was 2.6705. This specific result reinforces the broader pattern seen at the family level: adding dynamic channels, especially on the interface representation, increased training-set fit more than it increased robust held-out predictive signal.

5.7 Summary of empirical conclusions

The final evaluation supports four main empirical conclusions. First, the full graph view was consistently stronger than the interface graph view for the present problem setup. Second, the molecular-dynamics-derived feature construction used here did not provide a robust generalization advantage over the static representation. Third, within the dynamic GraphVAE branch, reconstruction objectives focused on dynamic targets were not helpful. Fourth, the strongest overall model was the direct supervised baseline on the full static graph, not the variational model.

These conclusions are more stable than the corresponding conclusions from earlier single-split experiments because they are based on repeated outer held-out evaluation over 52 configurations and 520 total fits. At the same time, the results remain specific to the implementation choices used here: short explicit-solvent trajectories, static residue graphs with time-aggregated annotations, a 128-node interface patch, and approximate protein-only inter-chain force proxies. The negative result for the molecular-dynamics-derived encoding should therefore be interpreted as a result about this particular representation strategy rather than as a universal statement that protein dynamics are irrelevant for affinity prediction.

Chapter 6

Discussion and Conclusions

6.1 Main findings

The final evaluation supports three main conclusions. First, the full residue graph is consistently stronger than the interface-only graph for the present pipeline. Second, the molecular-dynamics-derived feature set used here does not provide a robust held-out improvement over the static representation. Third, the strongest overall model is the direct supervised baseline on the full static graph rather than the GraphVAE.

These conclusions are stronger than the corresponding conclusions from the earlier exploratory runs because they are based on a matched 1437-complex subset, 52 model configurations, and 520 outer held-out fits. The main value of this design is that the factor-level conclusions do not depend on a single split.

6.2 Interpreting the full-versus-interface result

The full-versus-interface result is the strongest finding in the thesis. Every matched variational comparison favored the full graph, and the same direction held for the direct supervised baselines. The simplest interpretation is that the interface view is too restrictive in its current form. In the final retained set, 97.1% of interface graphs hit the hard cap of 128 nodes, so the interface representation was usually being truncated rather than simply focused.

This does not show that interface modelling is inherently weak. It shows that the specific interface construction used here was too information-limited to compete with the full graph on the matched PPB-Affinity subset.

6.3 Interpreting the negative dynamic result

The dynamic result needs narrower wording. The thesis does not show that dynamics are unimportant for protein–protein affinity. It shows that the specific dynamic encoding used here does not improve held-out prediction over a strong static graph.

That outcome is plausible given the implementation. The trajectories are short, the dynamic channels are aggressively summarized, temporal order is discarded, and the force features are approximate protein-only inter-chain proxies rather than exact decompositions from the original solvated simulation. The preparation pipeline also makes simplifying choices that matter for interpretation: missing residues are not rebuilt, non-protein components are removed before simulation, equilibration frames remain in the saved trajectories, and pH and temperature fall back to defaults when metadata are unavailable.

Taken together, these choices describe a coarse dynamic summary layer rather than a rich temporal representation. The negative result is therefore informative, but it is a result about this representation strategy, not about dynamics in general.

6.4 Why the supervised baseline won

The direct supervised baseline outperformed the best GraphVAE by a modest but consistent margin. The baseline spends all of its capacity on affinity prediction, whereas the GraphVAE must divide capacity between reconstruction, latent regularization, and optional supervision. On the present dataset, that trade-off did not pay off in held-out accuracy.

This does not make the GraphVAE useless. It still provides a structured latent representation and a clear route for future representation-learning work. It does mean that, for the present thesis question, the simpler model gives the stronger empirical answer.

6.5 What the train–test gaps show

The train–validation–test behaviour helps interpret the dynamic result. The unsupervised GraphVAE families mostly underfit. The semi-supervised dynamic families fit the training data more strongly, but they do not improve held-out performance and they show larger train–test gaps than the corresponding static families. The extra dynamic channels therefore look more like additional flexibility than additional robust signal.

6.6 Limitations

The final study uses a retained matched subset of 1437 complexes rather than the entire curated dataset of 2818 complexes. The simulations are short and simplified. The

graph topology is static. The interface representation is capped aggressively. The dynamic features are summary statistics rather than learned temporal representations. The evaluation is stronger than a single split, but it is still internal validation on one dataset family rather than an external benchmark.

These limitations do not invalidate the main conclusions, but they do narrow their scope. The thesis provides evidence about a specific and fully described MD-to-graph pipeline. It does not provide the final word on whether richer uses of molecular dynamics could help protein–protein affinity prediction.

6.7 Conclusion

The thesis asked whether short explicit-solvent molecular dynamics could improve residue-graph prediction of protein–protein binding affinity. For the implemented pipeline, the answer is no. The full graph is better than the interface patch, the present dynamic feature set does not improve robust held-out performance over the static graph, and the strongest overall model is the direct supervised baseline on the full static graph.

This is still a useful result. It replaces optimistic single-split impressions with a cleaner conclusion and points to the next technical questions: better dynamic representations, longer simulations, and less restrictive interface definitions.

Appendix A

Supplementary Technical Background

A.1 Molecular dynamics in plain terms

The core idea of molecular dynamics is simple: atoms are treated as particles whose positions change over time according to forces derived from a potential-energy function. If $\mathbf{r}_i$ is the position of atom i, then the force on that atom is

$$\mathbf{F}_i = -\nabla_{\mathbf{r}_i} U, \tag{A.1}$$

where U is the total potential energy of the system. In a biomolecular force field, this total energy is usually decomposed into bonded and nonbonded terms,

$$U = U_{\text{bond}} + U_{\text{angle}} + U_{\text{dihedral}} + U_{\text{elec}} + U_{\text{vdW}}. \tag{A.2}$$

The bonded terms maintain local covalent geometry, while the electrostatic and van der Waals terms model longer-range interactions between atoms that are not directly bonded.

In explicit-solvent simulation, water molecules and ions are included as ordinary particles in the simulation box. This is important because solvation contributes substantially to protein behaviour. The thermostat controls temperature, the barostat controls pressure, and periodic boundary conditions prevent the finite simulation box from behaving like an isolated vacuum droplet. Particle-mesh Ewald electrostatics are used because direct electrostatic summation over all periodic images would otherwise be too expensive for realistic systems.

For the present thesis, the goal of MD is not to compute a rigorous free energy of binding. Instead, short trajectories are used as a source of physically motivated descriptors. The intuition is that even a relatively short trajectory may reveal useful information about which residues remain hydrated, which contacts persist, and which parts of the

interface fluctuate more strongly. This is why the trajectory outputs are later reduced to residue- and edge-level summary statistics.

A.2 Residue graphs and graph neural networks in plain terms

A residue graph replaces thousands of atoms with a smaller and more interpretable set of residues. Each residue becomes a node, and edges connect residue pairs that are spatially close or interact strongly. This creates a graph that is small enough for efficient learning but still rich enough to preserve important structural context.

A graph neural network updates each node by repeatedly combining its current representation with information from neighbouring nodes. The general pattern is

$$\mathbf{h}_i^{(\ell+1)} = \phi^{(\ell)} \left(\mathbf{h}_i^{(\ell)}, \sum_{j \in \mathcal{N}(i)} \psi^{(\ell)}(\mathbf{h}_i^{(\ell)}, \mathbf{h}_j^{(\ell)}, \mathbf{e}_{ij}) \right). \tag{A.3}$$

The message function $\psi^{(\ell)}$ says how much node j should influence node i, the update function $\phi^{(\ell)}$ combines that information with the current state of node i, and the edge features $\mathbf{e}_{ij}$ allow the strength or type of connection to matter. After several layers, the node states are pooled to form a graph-level vector that can be used for prediction.

The specific convolution used in this thesis is GINE, which extends the graph isomorphism network family to edge-feature-aware message passing [21, 22]. Intuitively, this makes it possible for the model to distinguish not just *which* residues are neighbours, but *how* they are related through distance, interaction type, or trajectory-derived contact statistics.

A.3 Variational autoencoders in plain terms

A standard autoencoder learns two maps. The encoder compresses an input into a low-dimensional latent code, and the decoder tries to reconstruct the input from that code. A variational autoencoder goes one step further: instead of producing a single latent vector, the encoder produces the parameters of a probability distribution over latent vectors, usually a Gaussian with mean μ and variance σ^2. A latent vector is then sampled as

$$\mathbf{z} = \mu + \sigma \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I), \tag{A.4}$$

which is the reparameterization trick introduced by Kingma and Welling [15]. This trick allows the model to remain differentiable during training.

The VAE objective contains two parts. The reconstruction term says that the latent variable should preserve information needed to reconstruct the input. The Kullback–Leibler term says that the latent distribution should remain close to a simple prior, usually a standard normal distribution. If the KL term is too weak, the latent space can become irregular and hard to interpret. If it is too strong, the latent code can collapse and become uninformative. In practice, one often increases the KL weight gradually during training so that the model first learns to reconstruct and then learns to regularize its latent space.

In this thesis, the VAE is used as a representation-learning model on graphs. The encoder produces a graph-level latent code, and the decoder reconstructs masked node and edge features. The model is therefore not being asked to generate full protein complexes from scratch. Instead, it is being asked to learn a latent representation that is useful for reconstructing selected graph attributes and, in the semi-supervised setting, also useful for affinity prediction.

A.4 Why use ridge regression on the latent space

The final GraphVAE evaluation is performed with a ridge regressor on the exported latent means rather than with the neural affinity head alone. The reason is interpretability. If a very flexible nonlinear regressor were used after the latent space, it would be difficult to know whether good performance reflected a strong latent representation or simply a strong downstream regressor. Ridge regression is much simpler. It learns a linear mapping from latent space to affinity while penalizing large coefficients:

$$\hat{\mathbf{w}} = \arg\min_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \alpha\|\mathbf{w}\|_2^2. \tag{A.5}$$

If the latent space is genuinely informative, even this simple model should perform well. If it does not, then the representation itself is probably not strong enough.

This evaluation choice also makes comparisons cleaner. The direct supervised baseline answers the question "how well can affinity be predicted directly from the graph?" The ridge probe answers the question "how predictive is the representation learned by the VAE?" These are related but not identical questions, and it is useful to keep them separate.

A.5 Why repeated outer resampling matters in practical terms

Suppose two model configurations differ by only a small amount. On one train/test split, one of them may look better simply by chance. If the project stopped there, the resulting scientific story could be misleading. Repeated outer resampling reduces that risk.

A.5. Why repeated outer resampling matters in practical terms

Each configuration is retrained from scratch on multiple distinct train/validation/test partitions, and conclusions are based on the aggregate pattern across those runs.

In practical terms, this means that the thesis does not rely on one favourable experiment. Instead, it asks whether a design choice such as using the full graph, using dynamic features, or using a variational objective remains beneficial across many retrainings and many held-out subsets. This is why some results that looked promising in earlier one-split experiments no longer looked compelling in the final evaluation matrix.

Appendix B

Archived Approaches and Project Evolution

This appendix records the main branches that were explored seriously during the project but did not become part of the final reported pipeline. They are preserved under `ARCHIVE` in the public repository and can also be browsed on GitHub at `https://github.com/RohanDevereux/Bridging/tree/main/ARCHIVE`. The purpose of this appendix is to show how the final thesis question became narrower and more defensible.

B.1 Correction learning around fast affinity baselines

One early project idea was to avoid predicting binding affinity entirely from scratch. Instead, the model could start from a fast existing estimator and learn a correction term on top of it. This is an attractive strategy because it combines a hand-designed or physics-inspired baseline with a statistical model that may learn where the baseline fails. Two branches were explored in this spirit: a contact-based baseline using PRODIGY [9] and an endpoint free-energy baseline using MM/GBSA [8].

B.1.1 PRODIGY

The PRODIGY-related code is preserved under `ARCHIVE/PRODIGY`. This branch contains dataset parsing, chain-group normalization, estimate prefetching, and a runner that calls an external PRODIGY executable and parses the predicted ΔG. The motivation was straightforward: if a contact-based structural predictor already gives a rough estimate, perhaps a second model can learn the residual error more easily than predicting affinity from raw structural input alone. That idea is reasonable, and it is grounded in the literature behind PRODIGY itself [9].

In practice, this branch never became a strong thesis backbone. The preserved outputs are exploratory rather than part of the final evaluation, and the branch was operationally

awkward because many complexes failed during execution. More importantly, the baseline itself was not strong enough to make correction learning an attractive central story. For the final thesis, a direct comparison between static and MD-derived graph representations was easier to justify and easier to audit.

B.1.2 MM/GBSA

The MM/GBSA branch is preserved under `ARCHIVE/MMGBSA`. It includes AmberTools-based input preparation, HPC launchers, prefetched estimate handling, and a runner that can execute MM/GBSA from a structure or trajectory. The motivation was again correction learning, but with a stronger physics flavour. MM/GBSA occupies a useful middle ground in the literature: it is more physics-based than a purely empirical scorer, but much cheaper than rigorous free-energy calculation [8].

This branch was substantial from an engineering point of view, but it never matured into a clean final result set comparable to the graph-based matrix reported in the main body of the thesis. That mattered. By the time the project direction settled, the final thesis needed a central result that was easy to inspect, rerun, and explain directly from the repository. MM/GBSA did not meet that bar, and the approximation-dependence of MM/GBSA itself made it a difficult foundation for the final story [8].

B.2 The Cα contact-map branch

The oldest major modelling branch in the repository is the contact-map pipeline preserved under `ARCHIVE/CA_contact_map`. This branch converts selected interface residues into fixed two-dimensional tensors built from Cα–Cα contact and distance channels, then trains a convolutional variational autoencoder and downstream regression model on those tensors.

The motivation was straightforward. A contact map gives a fixed-size, image-like representation that is much easier to batch and process than a variable-size protein graph. That design choice has close relatives in protein–ligand learning, where voxelized or tensorized structural inputs have been used successfully [13]. For an early-stage project, this was a sensible way to get a learnable structural representation running quickly.

The reason this branch was eventually archived is also straightforward. Once the graph pipeline existed, the contact-map representation started to look too compressed. It discarded most residue identity information, side-chain detail, and explicit graph topology, while also depending heavily on patch selection and ordering conventions. The residue-graph formulation retained more structural information and connected more naturally to the MD-derived features that became central to the final project.

B.3 The PPB-inspired augmentation and spatiotemporal branch

The PPB-inspired deep-learning branch is preserved under `ARCHIVE/PPB_Affinity_augmentation`. It contains frame-level training, trajectory-level training, and an ensemble that combines predictions from several learned views, together with a substantial vendored model stack under `ARCHIVE/PPB_Affinity_augmentation/src/BaselineModel`.

The motivation here was different from the correction-learning branches. This path tried to use MD more directly by learning from frames and trajectories rather than reducing the trajectory to a small set of handcrafted graph features. That direction was reasonable. PPB-Affinity was explicitly introduced as a machine-learning resource for protein drug discovery [3], and recent protein–ligand work has shown that spatio-temporal learning on MD-derived data can be effective in some settings [14].

This branch was archived mainly because the project focus narrowed. It introduced a large amount of modelling machinery at once: frame sampling, temporal modelling, augmentation policy, ensembling, and a sizeable external code stack. The later MD-to-graph pipeline was narrower, easier to audit, and easier to connect to the final thesis questions. So this branch is better understood as a serious exploration that was superseded by a more focused direction, not as a simple failed experiment.

B.4 Exploratory dataset curation utilities

The final archived workstream is `ARCHIVE/Data_curation`. These scripts cover earlier filtering decisions, partner-size prefetching, subset selection, diverse-sample construction, and older cleaning steps for intermediate datasets.

This material matters because the final dataset definition was not chosen in a vacuum. Earlier in the project there was a real need to balance biological scope, computational tractability, and representation design. Once the final broad, pair-unique PPB-Affinity-derived dataset was fixed, these utilities were no longer part of the main pipeline, but they remain useful as provenance.

B.5 What the archived branches contributed

Although these branches did not survive into the final reported pipeline, they influenced the final thesis in concrete ways. The correction-learning branches clarified that fast structure-only or endpoint baselines were worth comparing against, but were not enough to anchor the whole project. The contact-map branch showed that a fixed tensor representation made the engineering easier at the cost of too much structural compression.

The PPB-inspired augmentation branch showed that direct frame- and trajectory-level learning was possible, but that it would be difficult to keep the final thesis as focused and auditable as it needed to be. The data-curation utilities helped narrow the problem to a dataset definition broad enough to be interesting and stable enough to support repeated evaluation.

In that sense, the archived material is not just dead code. It is the record of how the final thesis question became sharper. The thesis eventually settled on a narrower but more defensible comparison: static versus MD-derived residue-graph features on a matched protein–protein affinity dataset, evaluated under repeated held-out resampling.

Appendix C

Complete Model Matrix

This appendix gives the complete 52-model evaluation matrix, sorted by mean test RMSE. The machine-readable version is available in `RESULTS/final_resample_matrix_summary.csv`. Here *Sup.* denotes supervision status and *Gap* denotes the mean test-minus-train RMSE gap.

Table C.1: Complete 52-model evaluation matrix, sorted by mean test RMSE. Metrics are means over 10 outer held-out fits.

View	Model	Mode	Sup.	Target	z	RMSE	r	R^2	Gap
Full	Baseline	S	sup	–	–	2.3379	0.5064	0.2146	0.8021
Full	Baseline	SD	sup	–	–	2.4073	0.4802	0.1685	0.8414
Full	GraphVAE	S	semi	shared static	32	2.4292	0.4649	0.1537	0.7451
Full	GraphVAE	SD	semi	shared static	8	2.4364	0.4749	0.1478	1.1303
Full	GraphVAE	S	semi	shared static	8	2.4407	0.4507	0.1451	0.6921
Full	GraphVAE	S	semi	shared static	16	2.4407	0.4499	0.1438	0.6580
Full	GraphVAE	SD	semi	static+dynamic	16	2.4419	0.4503	0.1437	0.8952
Full	GraphVAE	SD	semi	shared static	32	2.4423	0.4669	0.1443	1.2169
Full	GraphVAE	SD	semi	shared static	16	2.4516	0.4629	0.1354	1.1312
Full	GraphVAE	SD	semi	static+dynamic	32	2.4687	0.4411	0.1233	0.8527
Full	GraphVAE	SD	semi	dynamic only	32	2.4740	0.4508	0.1231	1.0153
Full	GraphVAE	SD	semi	dynamic only	8	2.5168	0.4202	0.0920	0.9580
Full	GraphVAE	SD	semi	static+dynamic	8	2.5277	0.4092	0.0818	0.7819
Full	GraphVAE	SD	semi	dynamic only	16	2.5543	0.3954	0.0629	1.0628
Interface	Baseline	S	sup	–	–	2.5782	0.3451	0.0470	0.8090
Interface	Baseline	SD	sup	–	–	2.5829	0.3347	0.0463	0.6954
Full	GraphVAE	S	unsup	shared static	32	2.5906	0.2271	0.0401	0.0617
Interface	GraphVAE	S	semi	shared static	8	2.6041	0.3402	0.0282	0.8018
Full	GraphVAE	S	unsup	shared static	16	2.6052	0.1910	0.0303	0.0194
Full	GraphVAE	S	unsup	shared static	8	2.6069	0.1984	0.0286	0.0016
Interface	GraphVAE	S	semi	shared static	32	2.6113	0.3364	0.0207	0.7354
Interface	GraphVAE	S	semi	shared static	16	2.6180	0.3316	0.0166	0.7561
Interface	GraphVAE	SD	semi	static+dynamic	16	2.6238	0.3252	0.0146	0.7597
Full	GraphVAE	SD	unsup	shared static	16	2.6313	0.1259	0.0102	0.0265
Full	GraphVAE	SD	unsup	shared static	8	2.6322	0.1249	0.0103	0.0124
Interface	GraphVAE	S	unsup	shared static	32	2.6324	0.1211	0.0102	0.0118
Full	GraphVAE	SD	unsup	static+dynamic	32	2.6328	0.1255	0.0093	0.0610
Full	GraphVAE	SD	unsup	dynamic only	32	2.6353	0.1100	0.0074	0.0364
Full	GraphVAE	SD	unsup	shared static	32	2.6365	0.1813	0.0039	0.1021
Interface	GraphVAE	SD	semi	dynamic only	16	2.6366	0.3298	0.0032	0.9460
Interface	GraphVAE	S	unsup	shared static	8	2.6427	0.0790	0.0025	0.0062
Full	GraphVAE	SD	unsup	dynamic only	16	2.6441	0.0981	0.0013	0.0292
Interface	GraphVAE	S	unsup	shared static	16	2.6454	0.0774	0.0003	0.0157
Full	GraphVAE	SD	unsup	static+dynamic	8	2.6454	0.0719	0.0006	0.0048
Interface	GraphVAE	SD	semi	shared static	16	2.6455	0.3362	-0.0015	1.1836

View	Model	Mode	Sup.	Target	z	RMSE	r	R^2	Gap
Full	GraphVAE	SD	unsup	dynamic only	8	2.6461	0.0706	0.0000	0.0013
Interface	GraphVAE	SD	unsup	shared static	8	2.6473	0.0618	-0.0010	0.0129
Interface	GraphVAE	SD	unsup	static+dynamic	8	2.6481	0.0552	-0.0016	0.0056
Interface	GraphVAE	SD	unsup	shared static	32	2.6530	0.0896	-0.0051	0.0451
Interface	GraphVAE	SD	unsup	static+dynamic	32	2.6534	0.0268	-0.0056	0.0222
Full	GraphVAE	SD	unsup	static+dynamic	16	2.6542	0.0582	-0.0060	0.0286
Interface	GraphVAE	SD	unsup	dynamic only	8	2.6552	0.0293	-0.0071	0.0097
Interface	GraphVAE	SD	unsup	dynamic only	16	2.6554	0.0235	-0.0070	0.0151
Interface	GraphVAE	SD	semi	dynamic only	8	2.6557	0.3279	-0.0116	1.0207
Interface	GraphVAE	SD	unsup	static+dynamic	16	2.6557	0.0209	-0.0073	0.0127
Interface	GraphVAE	SD	unsup	shared static	16	2.6575	0.0395	-0.0088	0.0353
Interface	GraphVAE	SD	unsup	dynamic only	32	2.6589	0.0347	-0.0099	0.0347
Interface	GraphVAE	SD	semi	shared static	8	2.6684	0.3270	-0.0217	1.1249
Interface	GraphVAE	SD	semi	shared static	32	2.6705	0.3185	-0.0230	1.2466
Interface	GraphVAE	SD	semi	static+dynamic	8	2.6743	0.3039	-0.0243	0.8622
Interface	GraphVAE	SD	semi	dynamic only	32	2.6855	0.3076	-0.0322	1.0470
Interface	GraphVAE	SD	semi	static+dynamic	32	2.6974	0.2941	-0.0411	0.9357

Appendix D

Repository and Implementation Availability

The implementation for this thesis is available in the public repository `https://github.com/RohanDevereux/Bridging`. Rather than reproducing long source listings in the thesis itself, I refer the reader to the parts of the repository that correspond directly to the pipeline described in the main text.

The molecular-dynamics preparation and simulation code lives in `src/bridging/MD`. The graph-preparation code, including DeepRank2 integration, dynamic feature extraction, force-proxy construction, and graph-view materialization, lives in `src/bridging/graphvae/prep`. The model definitions, dataset preprocessing, supervised baseline, and latent-space regression code live in `src/bridging/graphvae/ml`. The command-line runners used to assemble the final pipeline live in `src/bridging/graphvae/runners`. Shared dataset parsing and affinity-conversion utilities live in `src/bridging/utils`.

The curated input datasets used by the pipeline are stored under `src/bridging/processedData`. The compact summary of the final 52-configuration evaluation matrix is stored in `RESULTS/final_resample_matrix_summary.csv`. Archived project branches are preserved under `ARCHIVE`; Appendix B explains their role in the evolution of the project.

Bibliography

[1] A. E. Modell, S. L. Blosser, and P. S. Arora, Trends in Pharmacological Sciences **37**, 702 (2016).

[2] Z. Guo and R. Yamaguchi, Frontiers in Bioinformatics **2**, 1065703 (2022).

[3] H. Liu, P. Chen, X. Zhai, K.-G. Huo, S. Zhou, L. Han, and G. Fan, Scientific Data **11**, 1316 (2024).

[4] J. Jankauskaitė, B. Jiménez-García, J. Dapkūnas, J. Fernández-Recio, and I. H. Moal, Bioinformatics **35**, 462 (2019).

[5] P. L. Kastritis, I. H. Moal, H. Hwang, Z. Weng, P. A. Bates, A. M. J. J. Bonvin, and J. Janin, Protein Science **20**, 482 (2011).

[6] G. M. Torrie and J. P. Valleau, Journal of Computational Physics **23**, 187 (1977).

[7] Z. Cournia, B. Allen, and W. Sherman, Journal of Chemical Information and Modeling **57**, 2911 (2017).

[8] S. Genheden and U. Ryde, Expert Opinion on Drug Discovery **10**, 449 (2015).

[9] A. Vangone and A. M. J. J. Bonvin, eLife **4**, e07454 (2015).

[10] I. H. Moal, R. Agius, and P. A. Bates, Bioinformatics **27**, 3002 (2011).

[11] J. Xie, Y. Zhang, Z. Wang, X. Jin, X. Lu, S. Ge, and X. Min, BMC Bioinformatics **26**, 116 (2025).

[12] R. Wang, X. Fang, Y. Lu, and S. Wang, Journal of Medicinal Chemistry **47**, 2977 (2004).

[13] M. M. Stepniewska-Dziubinska, P. Zielenkiewicz, and P. Siedlecki, Bioinformatics **34**, 3666 (2018).

[14] P.-Y. Libouban, C. Parisel, M. Song, S. Aci-Sèche, J. C. Gómez-Tamayo, G. Tresadern, and P. Bonnet, Bioinformatics **41**, btaf429 (2025).

[15] D. P. Kingma and M. Welling, *Auto-encoding variational bayes*, arXiv preprint arXiv:1312.6114 (2014), URL `https://arxiv.org/abs/1312.6114`.

[16] R. Gómez-Bombarelli, J. N. Wei, D. Duvenaud, J. M. Hernández-Lobato, B. Sánchez-Lengeling, D. Sheberla, J. Aguilera-Iparraguirre, T. D. Hirzel, R. P. Adams, and A. Aspuru-Guzik, ACS Central Science **4**, 268 (2018).

[17] T. N. Kipf and M. Welling, arXiv preprint arXiv:1611.07308 (2016), URL `https://arxiv.org/abs/1611.07308`.

[18] M. Simonovsky and N. Komodakis, International Conference on Artificial Neural Networks pp. 412–422 (2018).

[19] K. Henzler-Wildman and D. Kern, Nature **450**, 964 (2007).

[20] G. Crocioni, D. L. Bodor, C. Baakman, F. M. Parizi, D.-T. Rademaker, G. Ramakrishnan, S. A. van der Burg, D. F. Marzella, J. M. C. Teixeira, and L. C. Xue, Journal of Open Source Software **9**, 5983 (2024).

[21] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, in *International Conference on Learning Representations* (2019), URL `https://openreview.net/forum?id=ryGs6iA5Km`.

[22] W. Hu, B. Liu, J. Gomes, M. Zitnik, P. Liang, V. Pande, and J. Leskovec, International Conference on Learning Representations (2020), URL `https://openreview.net/forum?id=HJlWWJSFDH`.

[23] R. Kohavi, in *Proceedings of the 14th International Joint Conference on Artificial Intelligence* (1995), pp. 1137–1143.

[24] OpenMM Developers, *Pdbfixer*, `https://github.com/openmm/pdbfixer` (2026), accessed 2026-03-17.

[25] P. Eastman, R. Galvelis, R. P. Peláez, C. R. A. Abreu, S. E. Farr, E. Gallicchio, et al., The Journal of Physical Chemistry B **128** (2024).

[26] J. A. Maier, C. Martinez, K. Kasavajhala, L. Wickstrom, K. E. Hauser, and C. Simmerling, Journal of Chemical Theory and Computation **11**, 3696 (2015).

[27] L.-P. Wang and T. J. Martinez, The Journal of Physical Chemistry Letters **5**, 1885 (2014).

[28] R. T. McGibbon, K. A. Beauchamp, M. P. Harrigan, C. Klein, J. M. Swails, C. X. Hernández, C. R. Schwantes, L.-P. Wang, T. J. Lane, and V. S. Pande, Biophysical Journal **109**, 1528 (2015).

[29] M. Fey and J. E. Lenssen, ICLR Workshop on Representation Learning on Graphs and Manifolds (2019), URL https://arxiv.org/abs/1903.02428.

[30] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al., Journal of Machine Learning Research **12**, 2825 (2011), URL https://jmlr.org/papers/v12/pedregosa11a.html.